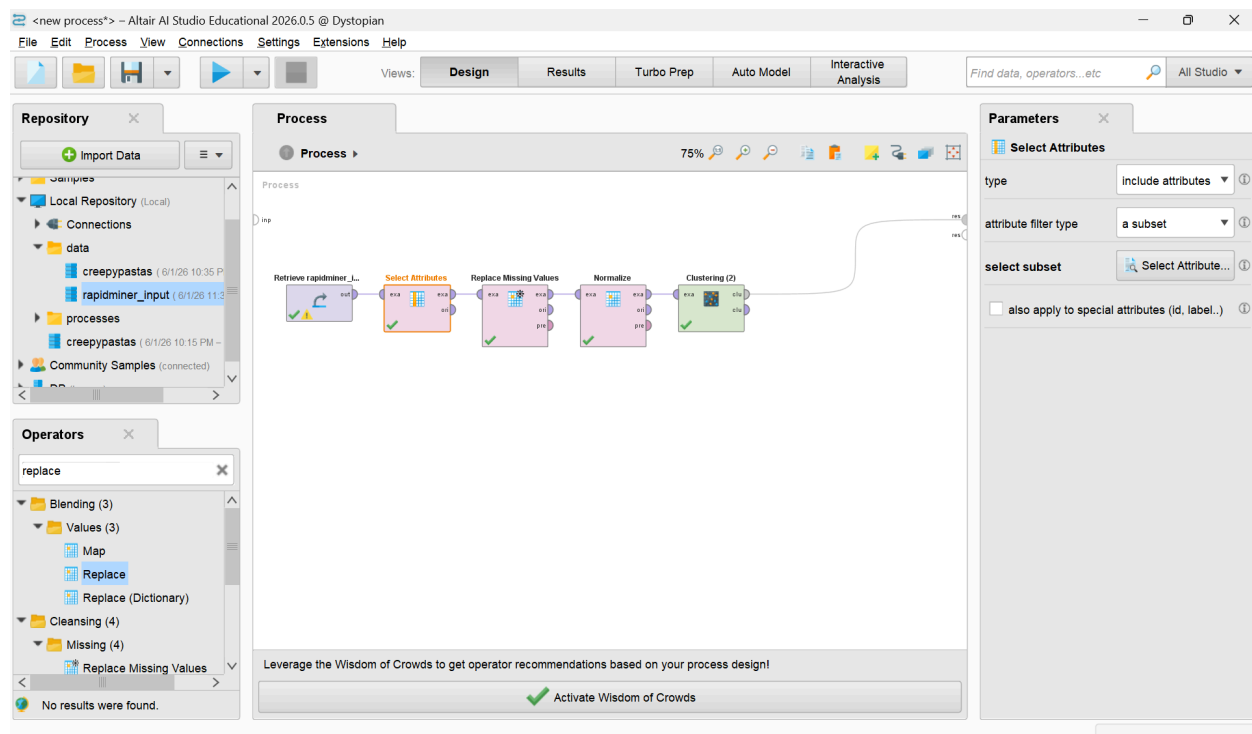


K-Means Clustering in RapidMiner

The process I followed involved retrieving from my RapidMiner_input dataset. Then, I selected the two numerical categories (those being reading_minutes and average_rating), before replacing all missing values to prevent errors in clustering. I then normalized the data before clustering it.

The K-Means procedure was carried out using two normalized numeric attributes: average_rating and reading_minutes. The model was configured to produce three clusters, and the resulting partition assigned all 3,509 stories to one of these groups. The centroid values were then examined to determine the distinguishing characteristics of each cluster. This allowed for a structural interpretation of the dataset based on length and reception.

Process:



The K-Means clustering model was used on the cleaned creepypasta dataset, considering the two numeric attributes average_rating and reading_minutes. This algorithm was able to divide the 3,509 pieces into three clusters:

Cluster 0: 2,397 pieces

Cluster 1: 1,019 pieces

Cluster 2: 93 pieces

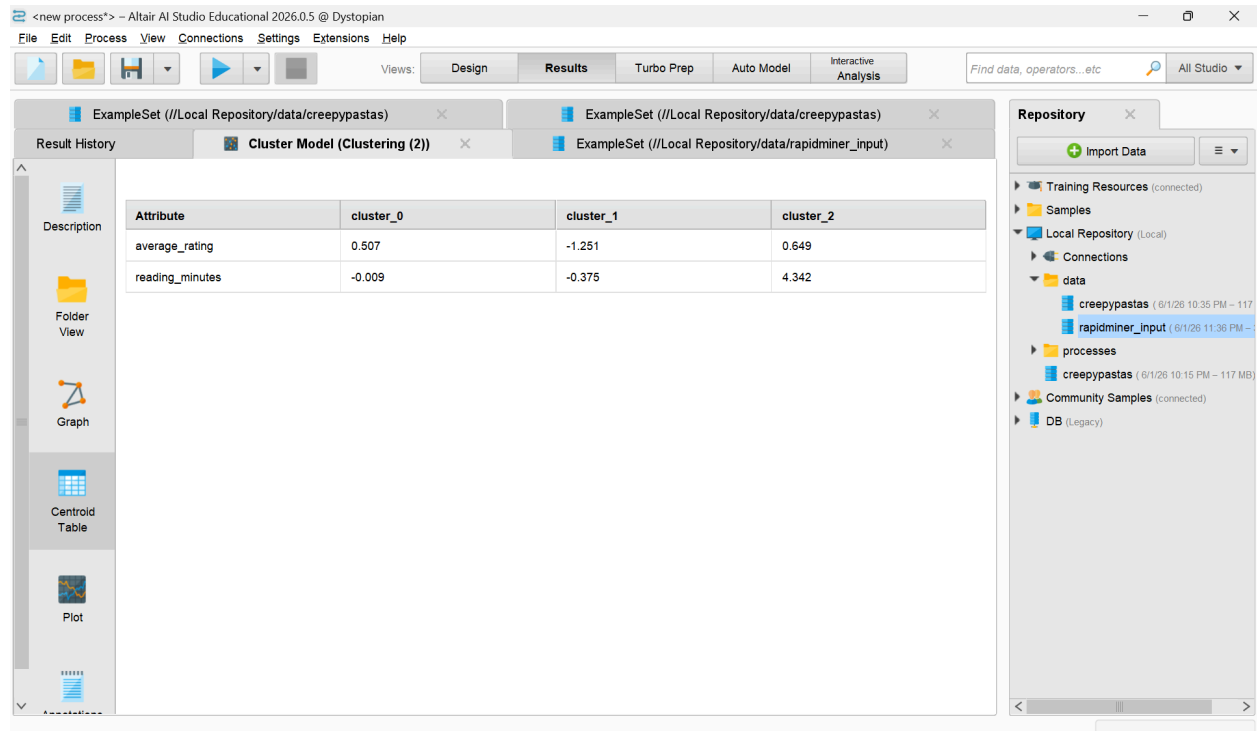
From the results above, it can be observed that there is a large central cluster, while there are also two other clusters that have unique features in terms of story length and average rating. The next step will be to examine the centroids to understand how these clusters were made. The K-Means clustering model was used on the cleaned creepypasta dataset, considering the two numeric attributes `average_rating` and `reading_minutes`.

The screenshot displays the Altair AI Studio Educational 2026.0.5 interface. The main window shows a 'Cluster Model (Clustering (2))' result. The 'Description' tab is active, displaying the following information:

- Cluster 0: 2397 items
- Cluster 1: 1019 items
- Cluster 2: 93 items
- Total number of items: 3509

The interface includes a sidebar with icons for 'Description', 'Folder View', 'Graph', 'Centroid Table', and 'Plot'. The top menu bar contains 'File', 'Edit', 'Process', 'View', 'Connections', 'Settings', 'Extensions', and 'Help'. The bottom right pane shows the 'Repository' view, listing various data sources and connections.

Cluster Interpretations



Attribute	cluster_0	cluster_1	cluster_2
average_rating	0.507	-1.251	0.649
reading_minutes	-0.009	-0.375	4.342

Cluster 0 - Cluster 0 is the largest cluster in the dataset, and it is the baseline for interpretation. Due to the normalization of the centroid values, a positive number indicates an above-average value, and vice versa, a negative number represents a below-average value. Cluster 0 has a rating of about 0.507 for `average_rating`, which means that the stories that comprise Cluster 0 are rated slightly higher than the mean of the dataset. However, the value of -0.009 given to `reading_minutes` means that the stories that make up this cluster are barely below the mean of the dataset. Altogether, these numbers suggest that Cluster 0 makes up roughly the average creepypasta story: not too highly rated, and almost average reading time.

Cluster 1 - Cluster 1 has negative numbers for both values, meaning that both `average_rating` and `reading_minutes` are below the mean of the dataset. Since the rating value is -1.251, this indicates that the ratings of this cluster are significantly lower than the mean of the dataset. Furthermore, the `reading_minutes` have a value of -0.375, meaning that the length of these stories is slightly shorter. Given this, there may be a correlation between shorter stories and lower ratings.

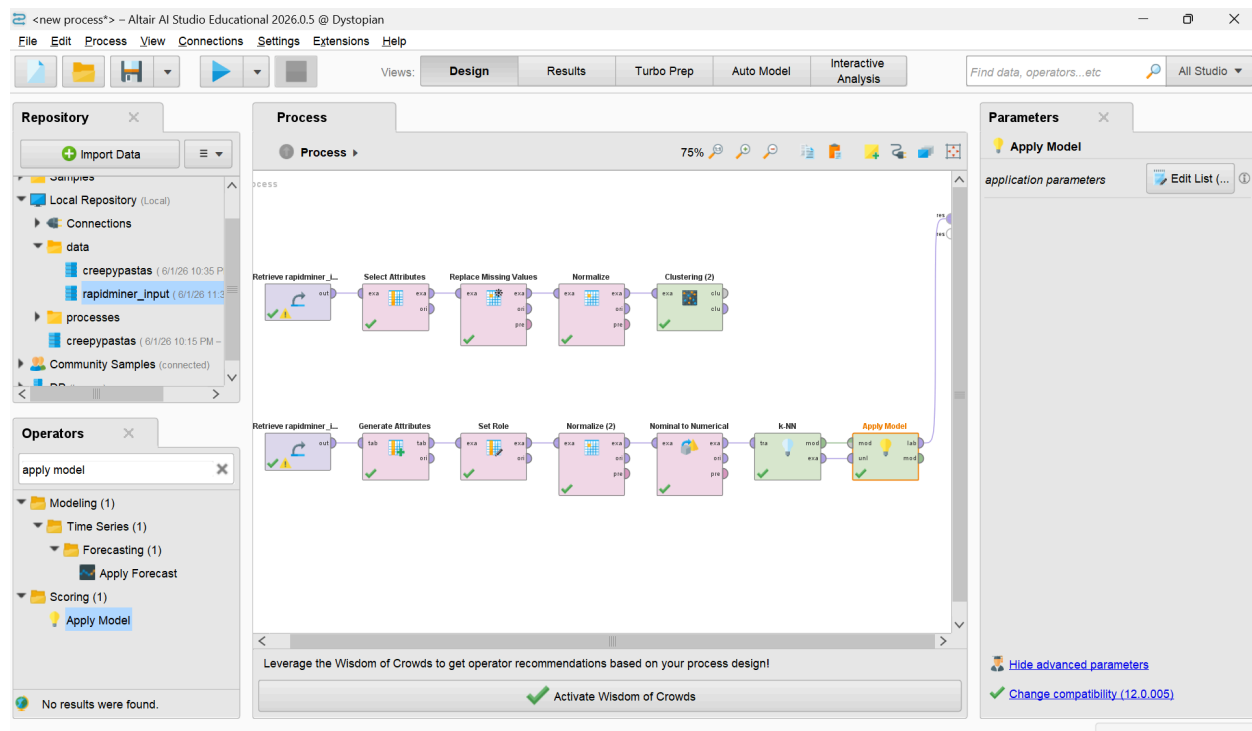
Cluster 2 - Cluster 2 has a positive number for both metrics (average_rating and reading_minutes), with its value for reading_minutes being 4.342. It stands in direct contrast to Cluster 1, and even to the dataset as a whole. Its values rise sharply above the mean, with the reading-time attribute extending far beyond any other group. This is substantially higher than the mean, suggesting that this cluster contains stories that are much longer than the average creepypasta story. Its average_rating of 0.649 is the highest of the clusters, meaning that these stories have the best reader responses and ratings. Therefore, cluster 2, although it has the fewest items, shows that longer narratives are best received by the audience. This suggests that readers may reward authors for longer, more developed stories.

Overall, the three clusters reveal a pattern within the data. Most stories fall into an average group, which is moderately rated and slightly longer than the mean. There is a smaller, yet still large group of shorter stories that tend to be rated lower than the average story. However, there is a very small group of longer narratives that tend to be rated positively by readers. These patterns show that story length and reader rating interact strongly, and that K-Means successfully separates the dataset into meaningful categories based on these two numeric features.

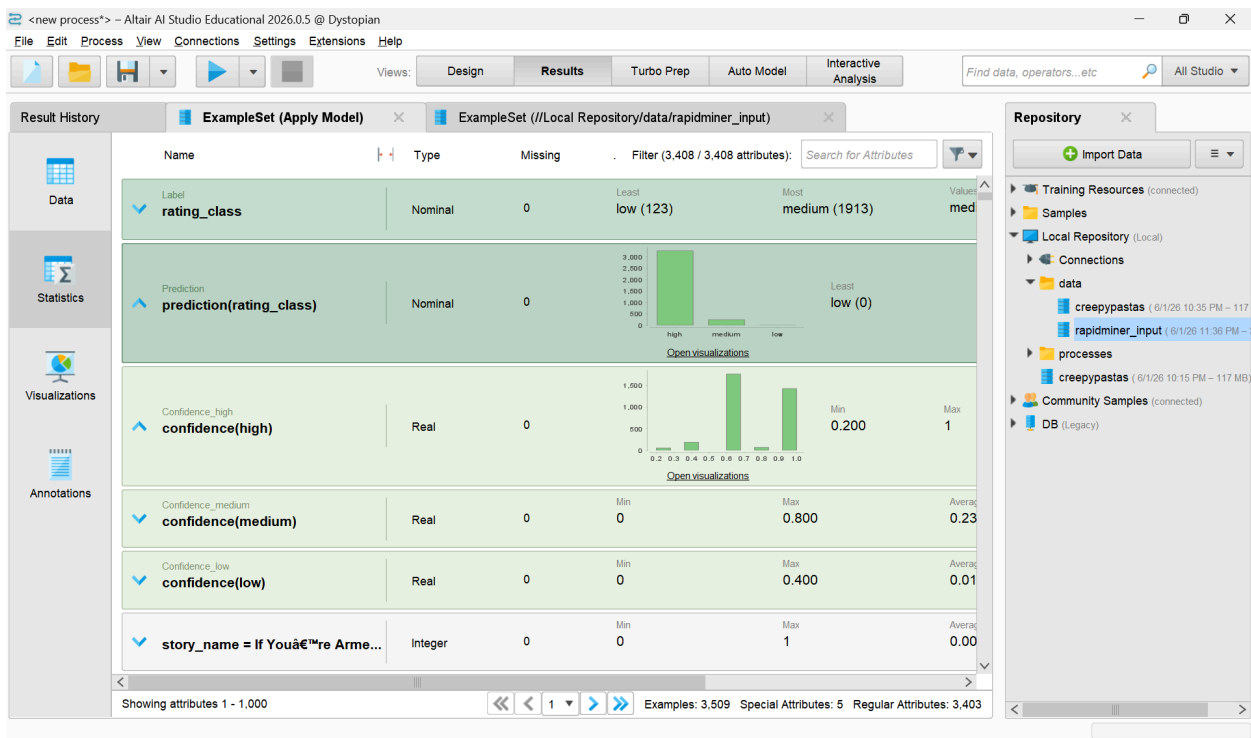
K-Nearest Neighbour Classification

The K-Nearest Neighbors stage of the analysis shifts from the unsupervised logic of clustering to a supervised framework in which the model must approximate a discretized rating label. After completing the preprocessing pipeline (normalizing the numeric predictors and generating the categorical label `rating_class`), I trained a KNN classifier with $k = 5$. This k value was chosen due to local sensitivity and overall stability. A small neighbourhood may have resulted in too much noise, but a too-large neighbourhood may have led to the collapse of meaningful distinctions and features. Normalization was necessary in the process because KNN uses distance calculations, so a lack of scale would have led to distortions within the dataset. Once the model was trained, I applied it to the dataset to obtain predicted labels and class-confidence values. I used the performance operator to retrieve the confusion matrix and other applicable metrics. This output provides a direct view of how the classifier behaves across the three discretized categories: high, medium, and low.

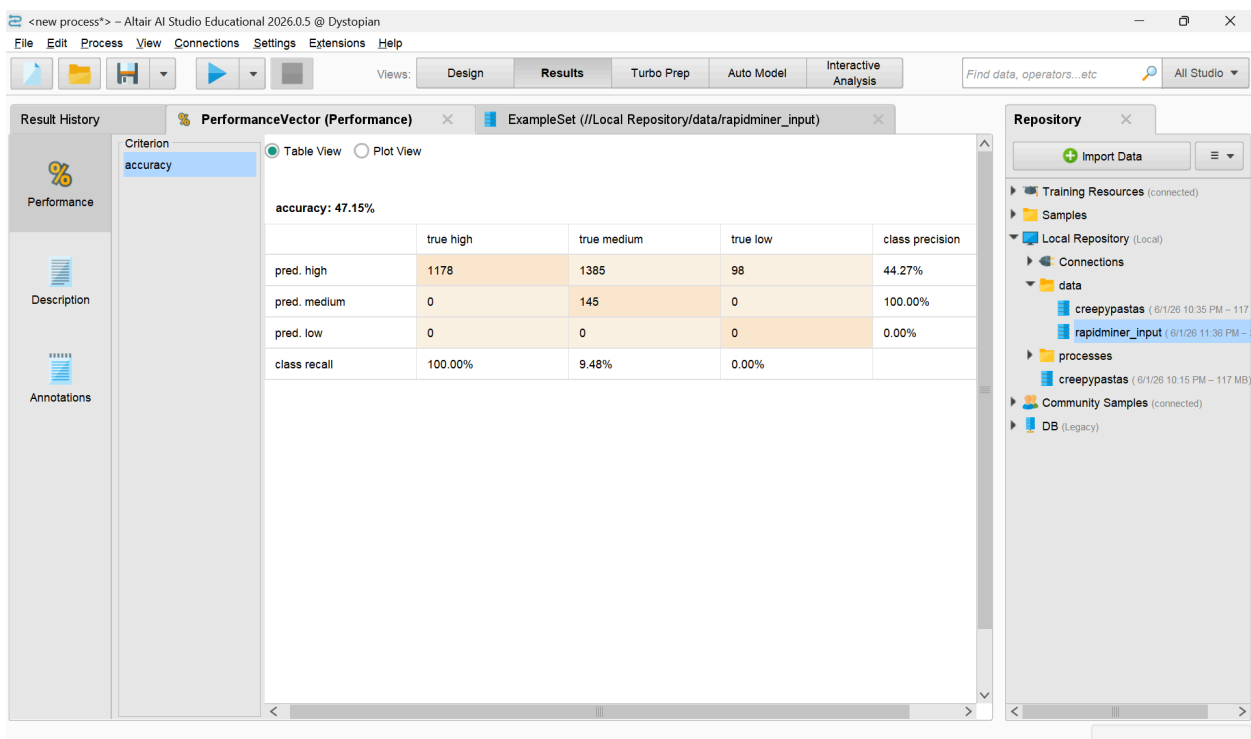
Process:



Results:



Confusion Matrix:



KNN Conclusions:

In this case, the confusion matrix displays an intrinsically biased model favoring the majority class. High values are predicted for almost all the samples, providing the classifier with a perfect recall (100%) for the class at the expense of a lower precision (44.27%). The same is true about the medium class: when the prediction for a particular instance equals 'medium', the classification result turns out to be 100% accurate, but the classifier does not predict 'medium' often enough – only in 9.48% of cases. Finally, the low class gets no correct predictions whatsoever (precision and recall are equal to zero). Overall accuracy stands at 47.15%.

It should be noted that this behavior is completely expected given the geometric characteristics of the problem in question. In other words, the discrete rating classes do not form clearly separated clusters within the feature space normalized for each individual dimension, and the strictly local KNN algorithm cannot determine any boundaries between the classes if there are none in the data provided. Therefore, the default classification results demonstrate that the available predictors contain insufficient information for distinguishing between medium- and low-rated stories.

In such a way, the obtained KNN results have a certain value. Indeed, they confirm the hypothesis that discretizing the rating attribute provides categories that are not well supported by numerical features, and thus, a new model should be built to obtain any separation. It is obvious from the confusion matrix that the classifier works properly only on those intervals that are well-filled, failing completely where there are few points.